Εργαστήριο 1

Αναγνώριση φωνής με Κρυφά Μαρκοβιανά Μοντέλα και Αναδρομικά Νευρωνικά Δίκτυα

Κόρδας Νικόλαος (03121032)
Κριθαρίδης Κωνσταντίνος (03121045)

November 4, 2025

Περιγραφή

Ο μηχανισμός παραγωγής φωνής

Φωνητική οδός (vocal tract): Πρόκειται για τη σειρά των οργάνων στον ανθρώπινο λαιμό και το κεφάλι που συνδέονται με την παραγωγή ήχου, από το λάρυγγα μέχρι τα χείλη και τη μύτη. Ο αρχικός ήχος παράγεται από την δόνηση των φωνητικών χορδών στον λάρυγγα. Ο ήχος ταξιδεύει μέσω της φωνητικής οδού. Το φάσμα της φωνής διαμορφώνεται από τη συχνοτική επιλεκτικότητα της φωνητικής οδού.

Formants: Οι συχνότητες συντονισμού του φωνητικού σωλήνα ονομάζονται formants.
Όταν μετράμε τα formants για τα φωνήεντα α, ου, ι, στην πραγματικότητα μετράμε τις συχνότητες συντονισμού του φωνητικού σωλήνα για το καθένα τους. Δεδομένου ότι ο φωνητικός σωλήνας των ανδρών είναι γενικά μεγαλύτερος από αυτόν των γυναικών, τα formants του ομιλητή 1 αναμένονται σε χαμηλότερες συχνότητες.

Pitch: Είναι η θεμελιώδης συχνότητα ταλάντωσης των φωνητικών χορδών κατά την παραγωγή του ήχου. Συνήθως, είναι χαμηλότερη στους άνδρες ενώ επηρεάζεται από το συναισθηματισμό, το τόνο και την προσωδία της ομιλίας. [1]

Βήμα 1

Χρησιμοποιούμε το praat για να αναλύσουμε τα σήματα ήχου μας και να εξάγουμε τις πληροφορίες που ζητούνται στην εκφώνηση.

Αρχείο onetwothree1.wav

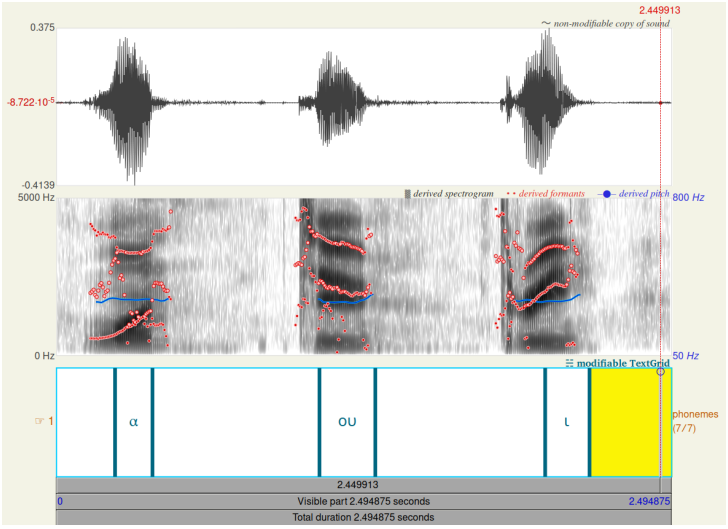
Ανοίγουμε το αρχείο στο praat για να κάνουμε annotations στα φωνήματα που μας ενδιαφέρουν. Έτσι, ορίζουμε ταυτόχρονα τις περιοχές όπου θα εξάγουμε το μέσο pitch. Πιο συγκεκριμένα, για αυτόν τον ομιλητή έχουμε ότι (από τα αρχεία που παράγει το praat):

- **Pitch (α):** 134.30091712757704 Hz (mean pitch in SELECTION)
- **Pitch (ου):** 130.34836851658105 Hz (mean pitch in SELECTION)
- **Pitch (ι):** 133.11983451151906 Hz (mean pitch in SELECTION)

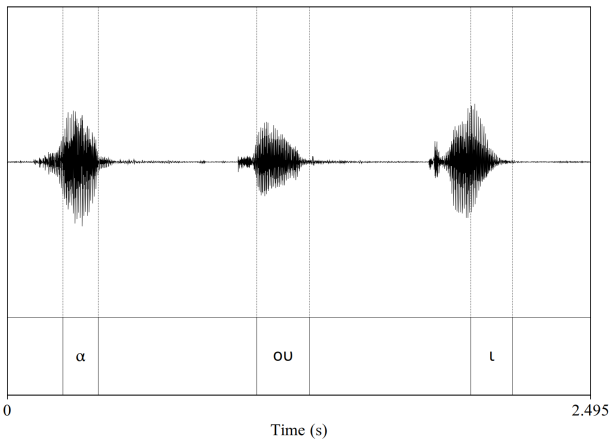
Αντίστοιχα, από το αρχείο για τα formants:

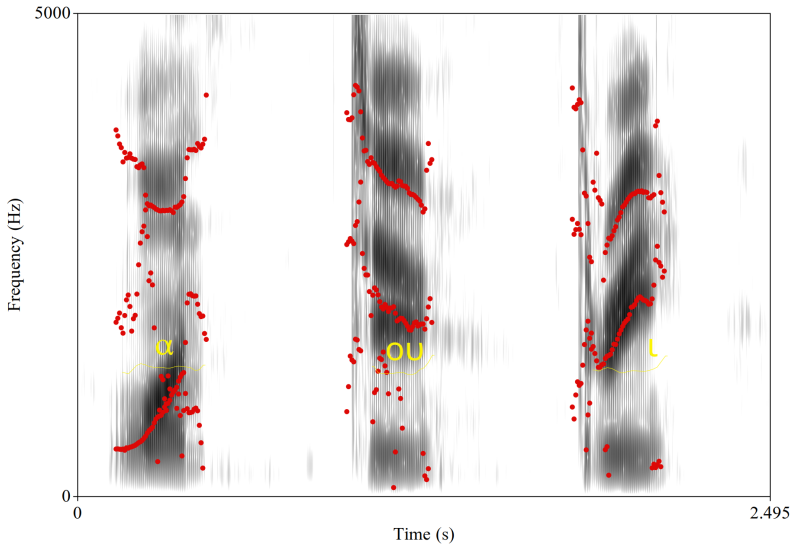
- **Φώνημα α:**
  - **F1_Hz:** 639.117504
  - **F2_Hz:** 3014.697815
  - **F3_Hz:** 3787.044970
- **Φώνημα ου:**
  - **F1_Hz:** 507.160260
  - **F2_Hz:** 2086.948531
  - **F3_Hz:** 3522.142285
- **Φώνημα ι:**
  - **F1_Hz:** 326.202470
  - **F2_Hz:** 2250.289657
  - **F3_Hz:** 3421.036991

Παρακάτω δίνουμε ένα στιγμιότυπο της διαδικασίας της επεξεργασίας και εξαγωγής των πληροφοριών:



Τέλος σχεδιάσαμε τα παρακάτω διαγράμματα που οπτικοποιούν την εργασία μας (1ο) και τις μετρήσεις μας (2ο)





Αρχείο onetwothree8.wav

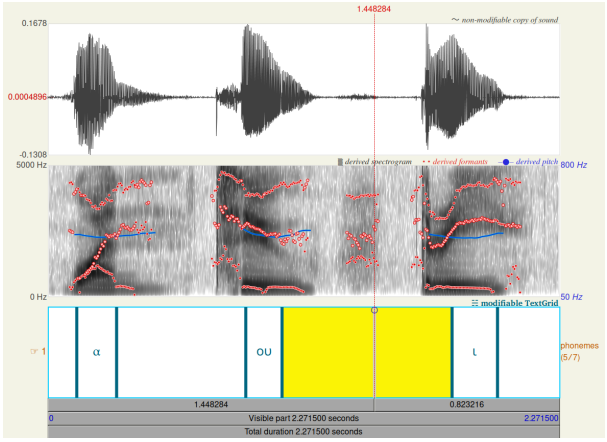
Αντίστοιχα με το προηγούμενο αρχείο οι μετρήσεις μας για το pitch κάθε φωνήματος είναι:

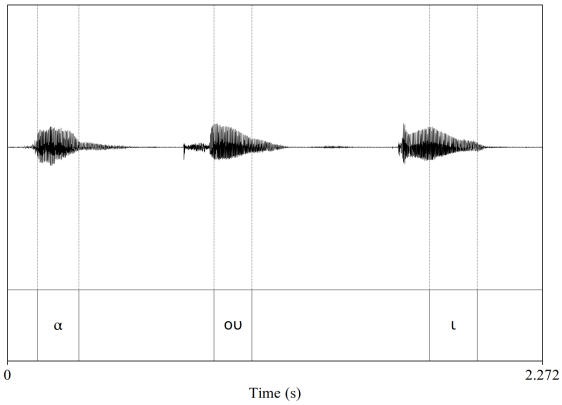
- **Pitch (α):** 178.90096253179337 Hz (mean pitch in SELECTION)
- **Pitch (ου):** 183.19462142657264 Hz (mean pitch in SELECTION)
- **Pitch (ι):** 176.88367983435367 Hz (mean pitch in SELECTION)

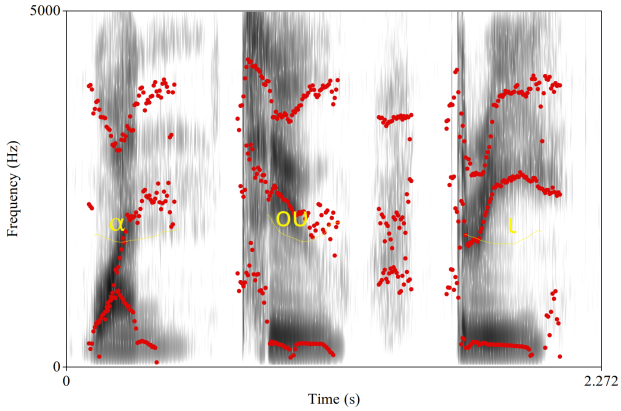
Για τα formants:

- **Φώνημα α:**
  - **F1_Hz:** 674.367922
  - **F2_Hz:** 703.507187
  - **F3_Hz:** 4112.994619
- **Φώνημα ου:**
  - **F1_Hz:** 376.486131
  - **F2_Hz:** 2801.849314
  - **F3_Hz:** 3860.898207
- **Φώνημα ι:**
  - **F1_Hz:** 345.351842
  - **F2_Hz:** 2549.549245
  - **F3_Hz:** 3396.739966

Τα αντίστοιχα διαγράμματα είναι:







Συμπεράσματα

Βλέπουμε πως οι μετρήσεις μας συμφωνούν απόλυτα με τις θεωρητικές προσδοκίες που αναφέρθηκαν στην περιγραφή παραπάνω. Πιο συγκεκριμένα, τα pitch (μέσες τιμές) της γυναίκας (αρχείο onetwothree8.wav) βρίσκονται σε υψηλότερες συχνότητες από τις αντίστοιχες του άντρα.

Επιπλέον, κατά κανόνα τα formants του άντρα βρίσκονται σε χαμηλότερες συχνότητες αντανακλώντας τα διαφορετικά φυσικά χαρακτηριστικά της φωνητικής του οδού (είναι μεγαλύτερη).

Βήμα 2

Στο βήμα αυτό ξεκινάει ο κώδικάς μας σε Python 3.
Ξεκινάμε γράφοντας την συνάρτηση data_parser που σκανάρει τον φάκελο `pr_lab2_data / digits` και επιστρέφει τρεις λίστες που περιέχουν τα δεδομένα των αρχείων wav, το id των ομιλητών και το ψηφίο που εκφωνούν, αντίστοιχα.
Τα δεδομένα του αρχείου wav τα φορτώνουμε με τη συνάρτηση load της βιβλιοθήκης librosa, τα ids των ομιλητών με την μέθοδο isdigit() των strings, και τα ψηφία με την μέθοδο isalpha() στο τμήμα του filename πριν την κατάληξη.

```
def data_parser(dir = 'pr_lab2_data/digits'):
    p = Path(dir)

    rosa_wavs = []
    speakers = []
    digits = []

    files = list(p.glob('*.wav')) # e.g. PosixPath('pr_lab2_data/digits/seven11.wav')

    for file in files:
        data, _ = librosa.load(file, sr=None)
        rosa_wavs.append(data)

        filename = file.parts[-1] # e.g. seven11.wav
        speakers.append(''.join([d for d in filename if d.isdigit()]))

        digits.append(''.join([char for char in filename.split('.')[-2] if char.isalpha()]))

    return rosa_wavs, speakers, digits
```

Στην συνάρτηση main καλούμε την συνάρτηση data_parser που μόλις γράψαμε ως εξής:

```
    # Step 2
    wavs, speakers, digits = data_parser()
    analyze_digits_statistics(digits)
```

Στο τέλος, όπως φαίνεται στον κώδικα παραπάνω, κάνουμε μία ανάλυση των στατιστικών των ψηφίων.

```
============================================================
ΣΤΑΤΙΣΤΙΚΑ ΨΗΦΙΩΝ ΑΠΟ ΤΑ ΔΕΔΟΜΕΝΑ
============================================================
Ψηφίο 'eight':  14 φορές ( 10.53%)
Ψηφίο 'five':  15 φορές ( 11.28%)
Ψηφίο 'four':  15 φορές ( 11.28%)
Ψηφίο 'nine':  15 φορές ( 11.28%)
Ψηφίο 'one':  15 φορές ( 11.28%)
Ψηφίο 'seven':  15 φορές ( 11.28%)
Ψηφίο 'six':  14 φορές ( 10.53%)
Ψηφίο 'three':  15 φορές ( 11.28%)
Ψηφίο 'two':  15 φορές ( 11.28%)
============================================================
Σύνολο δειγμάτων: 133
Μοναδικά ψηφία: 9
Πιο συχνό ψηφίο: seven (15 φορές)
============================================================
```

Αυτό το κάνουμε για να εξετάσουμε αν το dataset είναι ισορροπημένο ή όχι και αν όχι προς τα πού είναι biased. Όπως βλέπουμε, όμως, το dataset είναι ισορροπημένο. Έτσι, το accuracy θα είναι μία καλή μετρική για την αξιολόγηση των ταξινομητών που θα εκπαιδεύσουμε στη συνέχεια της εργασίας.

Βήμα 3

Στο βήμα αυτό εξάγουμε τα Mel-frequency cepstral coefficients (MFCCs) των σημάτων. Οι συντελεστές αυτοί προκύπτουν από Discrete Cosine Transform εφαρμοσμένο στον λογάριθμο της φασματικής πυκνότητας ισχύος ειλημμένη στην κλίμακα mel ώστε ισαπέχοντα διαστήματα συχνοτήτων να ακούγονται εξίσου μακριά στο ανθρώπινο αυτί. Η διαφορά στην κλίμακα είναι που την διαφοροποιεί από το κανονικό cepstrum, του οποίο η διαδικασία παριστάνεται και γραφικά στο παρακάτω διάγραμμα του [1].

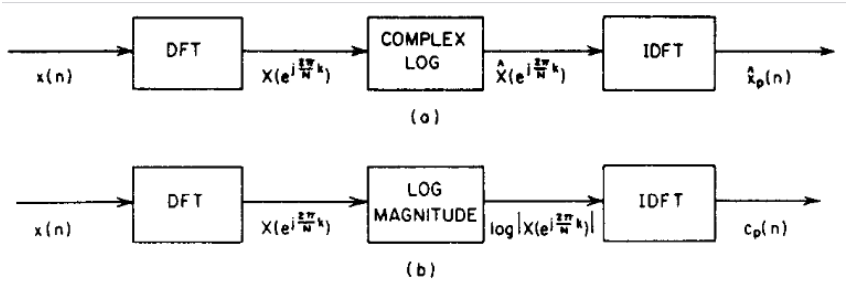


Fig. 7.7 Practical implementations of systems for obtaining (a) the complex cepstrum; and (b) the cepstrum.

Χρησιμοποιούμε παράθυρο μήκους 25ms, βήμα 10ms και εξάγουμε ως χαρακτηριστικά τα 13 MFCCs ανά αρχείο. Επιπλέον, υπολογίζουμε την πρώτη και δεύτερη διακριτή παράγωγο των MFCCs κατά την μετακίνηση του παραθύρου λήψης τους. Η βιβλιοθήκη librosa έχει συναρτήσεις για όλα αυτά τα ζητούμενα, οπότε η τελική συνάρτηση και η κλήση της στη main φαίνονται παρακάτω:

```
def extract_features(wavs):
    mfccs = []
    win_len = 25 * Fs // 1000
    step = 10 * Fs // 1000
    n_features = 13
    deltas = []
    delta_deltas = []

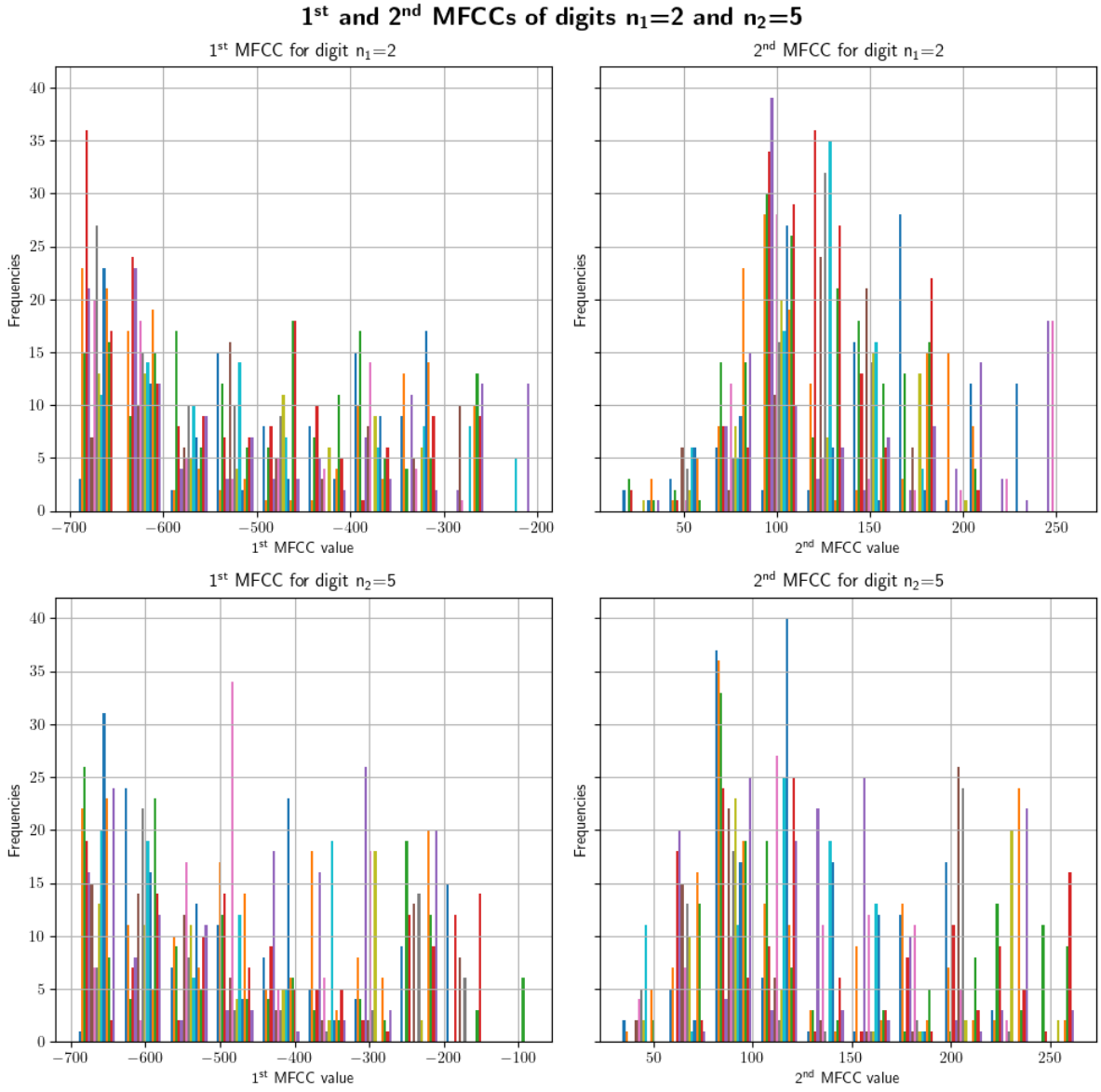
    for wav in wavs:
        mfcc = librosa.feature.mfcc(y=wav, sr=Fs, n_mfcc=n_features, hop_length
   =step, win_length=win_len)
        mfccs.append(mfcc)
        deltas.append(librosa.feature.delta(mfcc, order=1))
        delta_deltas.append(librosa.feature.delta(mfcc, order=2))

    return mfccs, deltas, delta_deltas
```

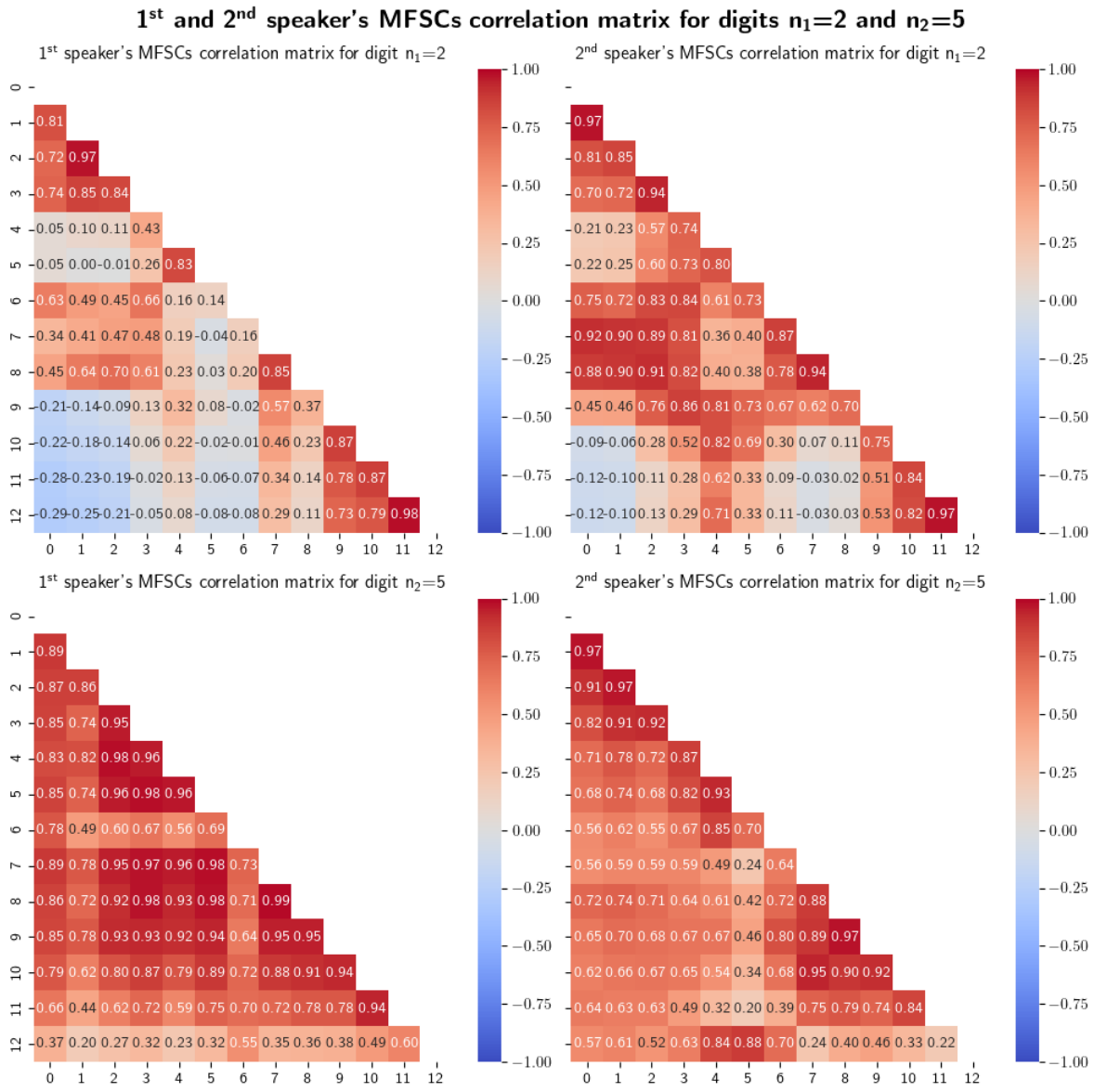
```
    # Step 3
    mfccs, deltas, delta_deltas = extract_features(wavs)
```

Βήμα 4

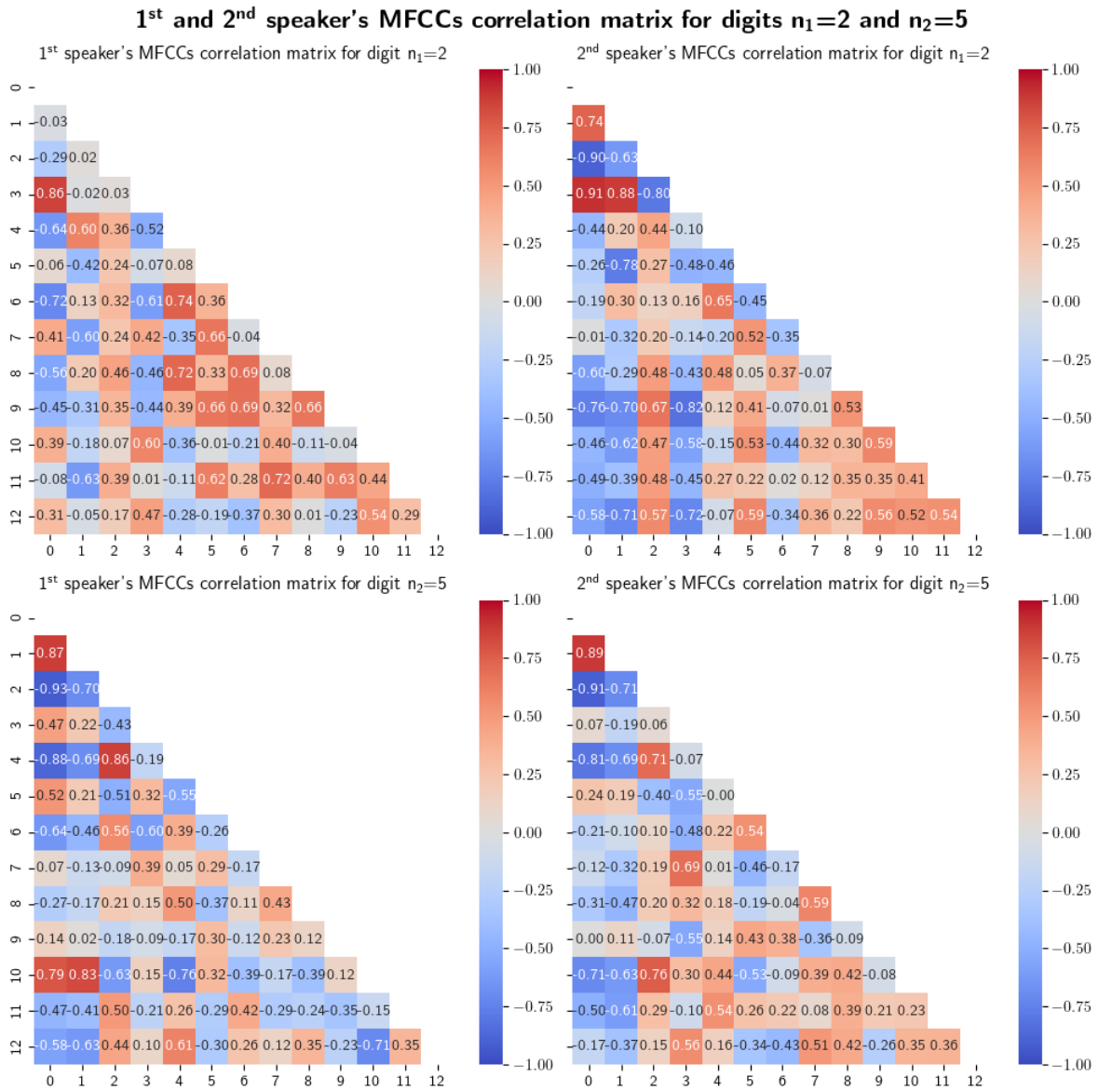
Αρχικά, δημιουργούμε ιστογράμματα για το 1ο και 2ο MFCC για όλες τις εκφωνήσεις των ψηφίων $n_1 = 2$ και $n_2 = 5$.



Αυτό που παρατηρούμε είναι ότι τα MFCC για $n_1 = 2$ δεν μοιάζουν πολύ μεταξύ τους στο ιστόγραμμα. Αυτό επιβεβαιώνεται και στο επόμενο γράφημα που αναπαριστά τη συσχέτιση μεταξύ τους, καθώς βλέπουμε ότι έχει τιμή 0,03 (γκρι χρώμα), για τον πρώτο ομιλητή. Από την άλλη πλευρά δεν παρατηρούμε το ίδιο για τον δεύτερο ομιλητή, πράγμα μη ευνοϊκό για το consistency της διαχωρισιμότητας των ψηφίων ανάμεσα στους ομιλητές. Αυτή τη παρατήρηση έρχονται να επιβεβαιώσουν και τα ιστογράμματα για $n_2 = 5$. Σε αυτά βλέπουμε ότι η μορφή τους μοιάζει αλλά και η συσχέτισή τους είναι υψηλή και για τους 2 ομιλητές (κοντά στο 0,85).
Επιπλέον, όσον αφορά την ικανότητα των MFCC να διαχωρίσουν τα διαφορετικά ψηφία, αυτό που μπορούμε να πούμε διαφάζοντας κατακόρυφα το διάγραμμα είναι ότι τα ιστογράμματα μοιάζουν μεν, δεν είναι πανομοιότυπα δε. Οπότε τα MFCC θα μπορούσαν να είναι η βάση του classification ειδικά εφόσον μετασχηματιστούν σε κάτι άλλο ή από αυτά εξαχθούν ορισμένα στατιστικά χαρακτηριστικά, πράγμα που κάνουμε παρακάτω με την μέση τιμή και την τυπική απόκλισή τους.
Στη συνέχεια, υπολογίζουμε τα Mel Filterbank Spectral Coefficients (MFSCs) για δύο εκφωνήσεις των ίδιων ψηφίων από δύο διαφορετικούς ομιλητές και υπολογίζουμε τον πίνακα συνδιακύμανσης τους για κάθε εκφώνηση με την συνάρτηση corrcoef της numpy, παρουσιάζοντας το τελικό αποτέλεσμα σε heatmap της βιβλιοθήκης seaborn.



Ύστερα, κάνουμε το ίδιο για τα MFCCs.



Παρατηρούμε ότι τα MFCCs έχουν πολύ μικρότερη συσχέτιση μεταξύ τους. Για αυτόν τον λόγο, όταν κάνουμε επιλογή χαρακτηριστικών, χρησιμοποιούμε τα MFCCs αντί τα MFSCs, αφού στους περισσότερους αλγορίθμους προτιμάμε τα χαρακτηριστικά μας να είναι κατά το δυνατόν ανεξάρτητα, και το να είναι ασυσχέτιστα είναι προϋπόθεση για αυτό το γεγονός.

Σημειώνουμε ότι τα MSFCs τα λάβαμε με την εντολή `librosa.feature.melspectrogram(y=wav, sr=Fs, win_length=win_len, hop_length=step, n_mels=n_features)`. Ισοδύναμα, θα μπορούσαμε να τα είχαμε λάβει με την εντολή `librosa.feature.inverse.mfcc_to_mel(mfcc)`, αφού η βιβλιοθήκη librosa έχει υλοποιημένη την μετατροπή MFCCs σε MFSCs, ή με την εντολή `librosa.db_to_power(scipy.fft.idct(mfcc, type=2, norm="ortho", axis=0))`, αφού τα MFSCs προκύπτουν με τον ίδιο ακριβώς τρόπο με τα MFCCs αλλά χωρίς την εφαρμογή του τελευταίου DFT, οπότε με Inverse DFS μπορούμε να τον αναιρέσουμε και να πάρουμε MFSCs από MFCCs. Την τριπλή αυτή ισοδυναμία επιβεβαιώσαμε και πειραματικά. Προτιμήσαμε τον πρώτο τρόπο, με το melspectrogram εφαρμοσμένο στο wav, γιατί εφαρμόζει τον ευθύ μετασχηματισμό, οπότε δεν υποφέρει από τα επιπλέον αριθμητικά σφάλματα που εισάγουν οι DCT και Inverse DCT.

Βήμα 5

Αρχικά χρησιμοποιούμε NumPy για να ενώσουμε όλα τα χαρακτηριστικά του input που έχουμε υπολογίσει (mfcc, delta, delta-deltas) και να φτιάξουμε έναν πίνακα. Κατόπιν υπολογίζουμε για καθένα από αυτά τη μέση τιμή και την τυπική απόκλιση και επιλέγουμε αυτά ως τα τελικά χαρακτηριστικά που θα χρησιμοποιήσουμε για το classification. Στα σχόλια έχουμε το shape των tensors που έχουμε εκτυπώσει για να είμαστε σίγουροι για την ορθότητα των βημάτων μας.

```
def combine_features(mfccs, deltas, delta_deltas, digits):
    statistical_characteristics = []

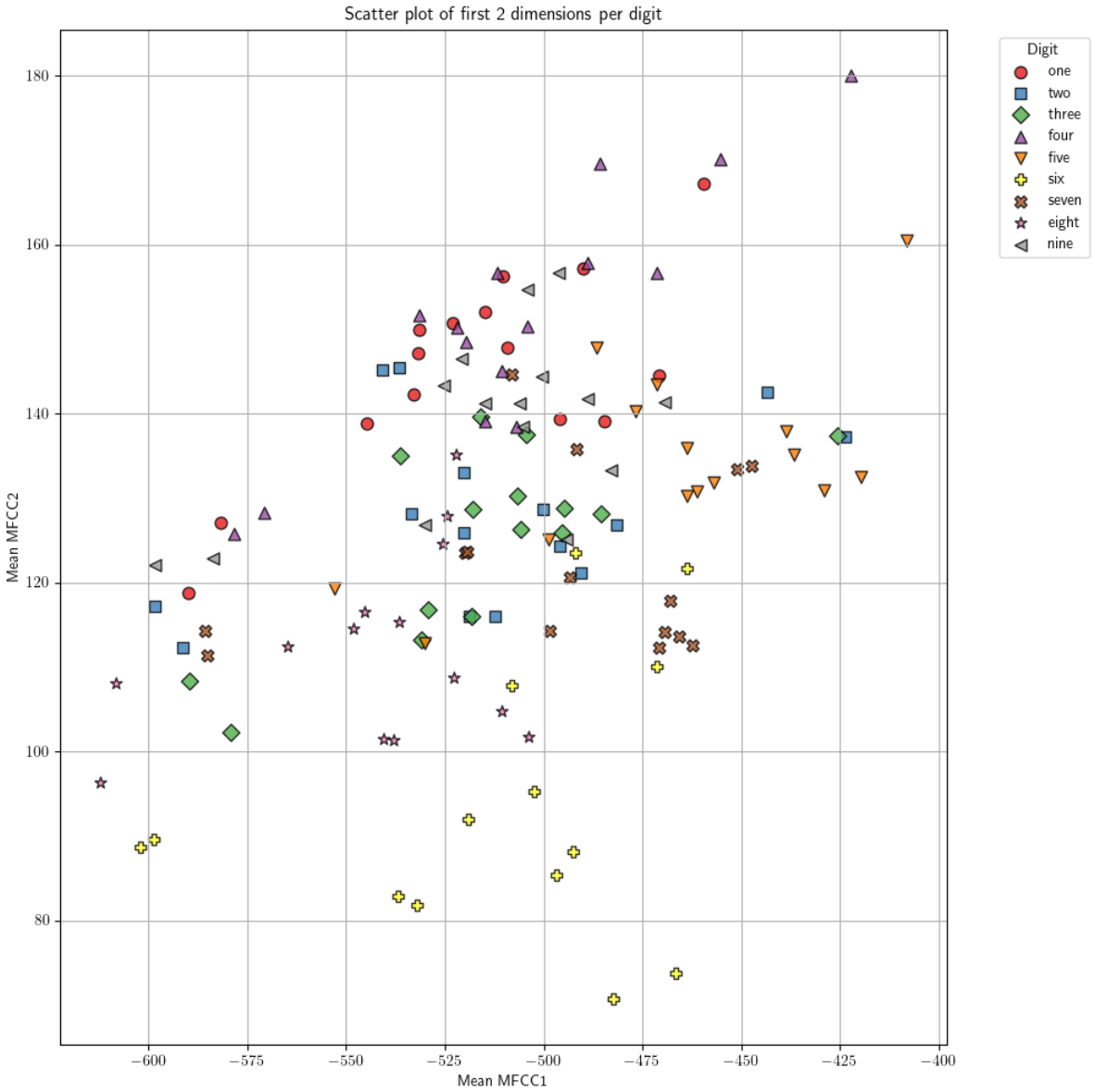
    for mfcc, delta, delta_delta in zip(mfccs, deltas, delta_deltas):
        features = np.vstack((mfcc, delta, delta_delta))   # shape -> (39, smth
    )

        mean = np.mean(features, axis=1) # axis=1 -> (39,)
        std = np.std(features, axis=1)

        characteristics = np.concatenate((mean, std)) # (78,)
        statistical_characteristics.append(characteristics)

    feature_vectors = np.array(statistical_characteristics)  # (133, 78)
```

Οι 2 πρώτες διαστάσεις που καλούμαστε να παραστήσουμε σε scatter plot είναι οι μέσες τιμές των πρώτων 2 MFCC. Το αποτέλεσμα φαίνεται παρακάτω (ο λόγος που χρησιμοποιούμε διαφορετικά χρώματα και σχήματα είναι για να κάνουμε τις κλάσεις μας ευδιάκριτες):

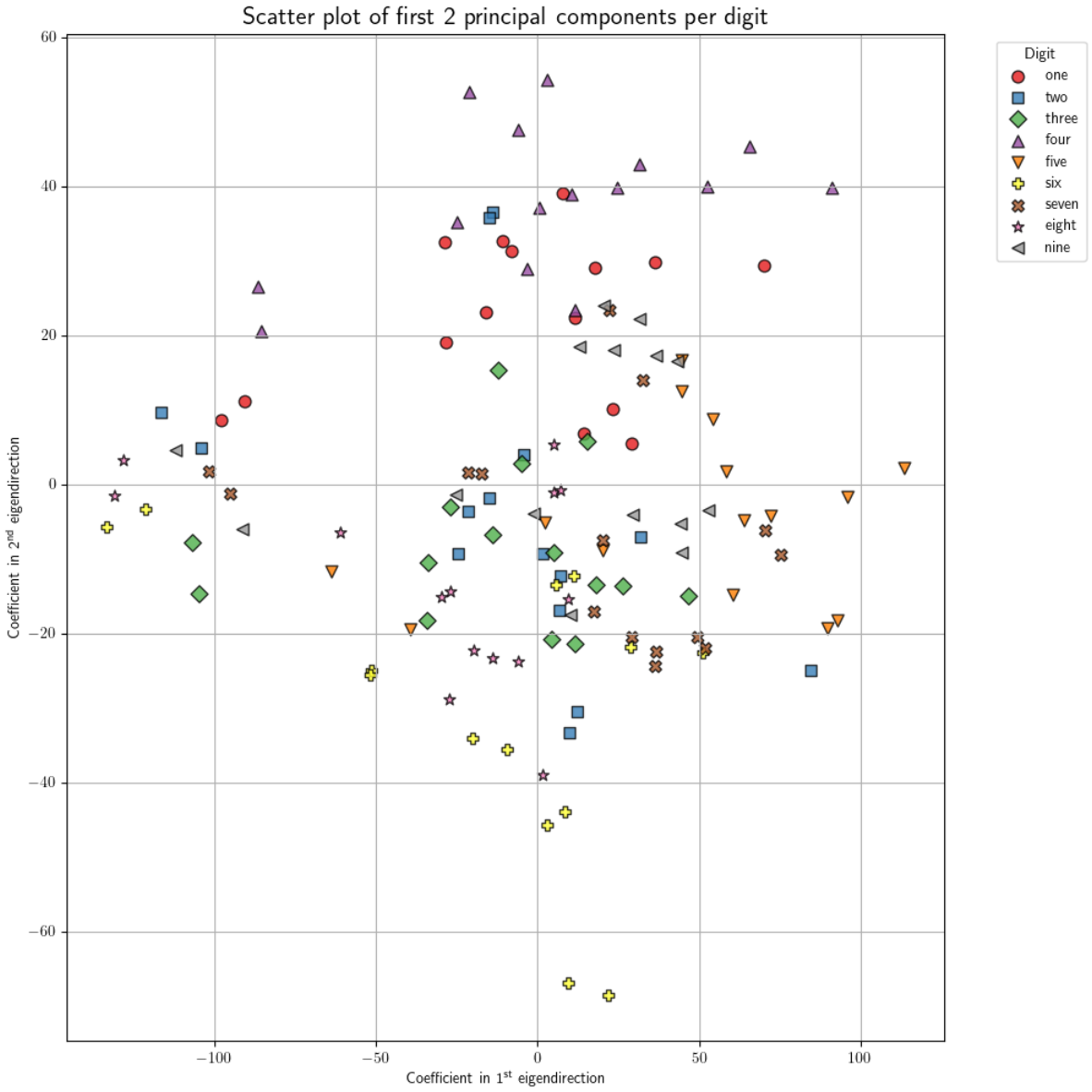


Ουσιαστικά, αυτό που κάνουμε είναι να βάζουμε 2 "τυχαία" (έστω τα 2 πρώτα) features στο scatter plot για να δούμε ότι οι κλάσεις μας δεν είναι τόσο διαχωρίσιμες. Προετοιμαζόμαστε να συγκρίνουμε το αποτέλεσμα με το PCA στο βήμα 6 όπου τα 2 πρώτα features γίνονται εκείνα που εξηγούν το μεγαλύτερο ποσοστό του variance και έτσι κάνουν τις κλάσεις μας πιο διαχωρίσιμες από αυτό που βλέπουμε παραπάνω.

Βήμα 6

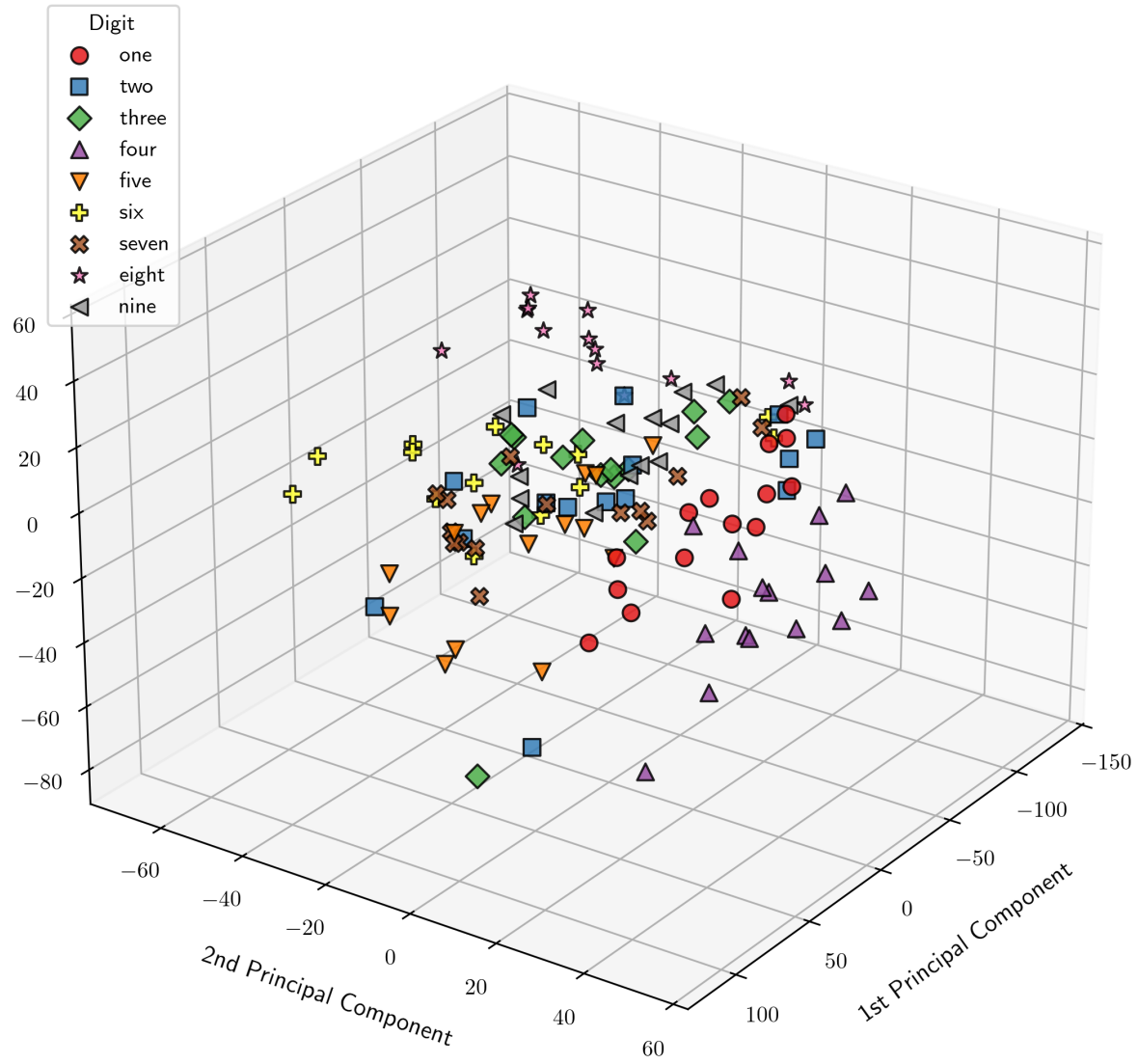
Στο βήμα αυτό, εφαρμόζουμε Principal Component Analysis (PCA) για την μείωση διαστατικότητας των δεδομένων. Αρχικά, μειώνουμε τις διαστάεις των διανυσμάτων χαρακτηριστικών σε 2 με την εντολή `PCA(n_components=2).fit_transform(feature_vectors)` και στη συνέχεια σε 3 θέτοντας n_components = 3.

Παρακάτω παραθέτουμε εκ νέου το scatter plot με τις δύο διαστάσεις των μετασχηματισμένων δεδομένων για n_components = 2.



Παρατηρούμε ότι τα σημεία ακόμα δεν είναι ιδιαίτερα διαχωρισμένα, αλλά πολύ περισσότερο από ό,τι στο Βήμα 5 χωρίς PCA. Τα ψηφία είναι καλύτερα μαζεμένα σε περιοχές, αλλά υπάρχουν πολλές επικαλύψεις ακόμα. Το ποσοστό της συνολικής ερμηνευόμενης διασποράς των δεδομένων είναι 70.5% (άθροισμα στοιχείων πεδίου explained_variance_ratio_).

Ακολούθως, παραθέτουμε το scatter plot με τις τρεις διαστάσεις των μετασχηματισμένων δεδομένων για n_components = 3.



Πλέον, παρατηρούμε ότι τα ψηφία είναι πολύ καλύτερα διαχωρισμένα. Είναι μαζεμένα σε συμπαγείς περιοχές και υπάρχουν πολύ λιγότερες επικαλύψεις, αν και ακόμα υπάρχουν. Επιπλέον, η συνολική ερμηνευόμενη διασπορά είναι 81.4%. Ακόμα, το πεδίο explained_variance_ratio_ μας δίνει το ποσοστό της ερμηνευόμενης διασποράς από κάθε συνιστώσα του PCA, δηλαδή τη διασπορά σε στον κάθε άξονα των πρώτων ιδιοδιανυσμάτων του πίνακα συνδιασποράς. Αυτή είναι πολύ μεγάλη για το πρώτο διάνυσμα και μονότονα μειώνεται όσο αυξάνονται οι συνιστώσες.

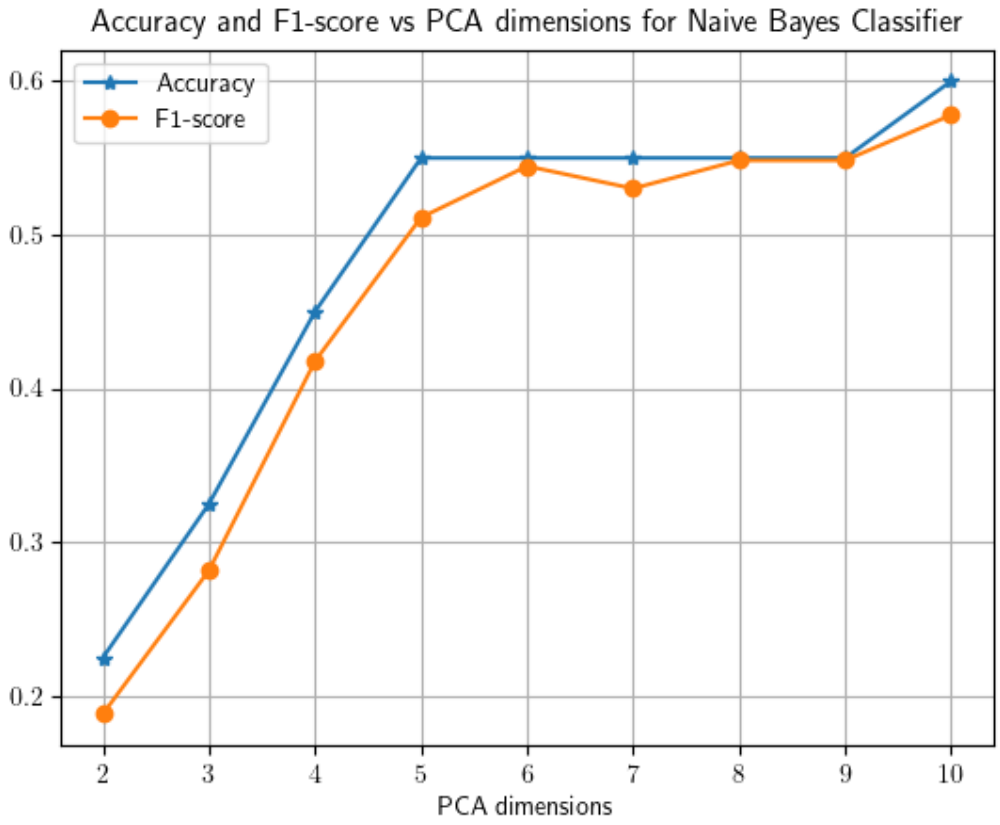
Μπορούμε να θεωρήσουμε ότι η μείωση διαστατικότητας ήταν επιτυχημένη, αφού ερμηνεύσαμε το 70.5% και 81.4% με μόνο 2 και 3 συνιστώσες αντίστοιχα, αλλά για να έχουμε ακόμα καλύτερη απόδοση, θα βοηθούσε να έχουμε λίγες ακόμα συνιστώσες ώστε να ερμηνεύσουμε ακόμα μεγαλύτερο τμήμα της διασποράς και να διαχωριστούν πλήρως τα δεδομένα.

Βήμα 7

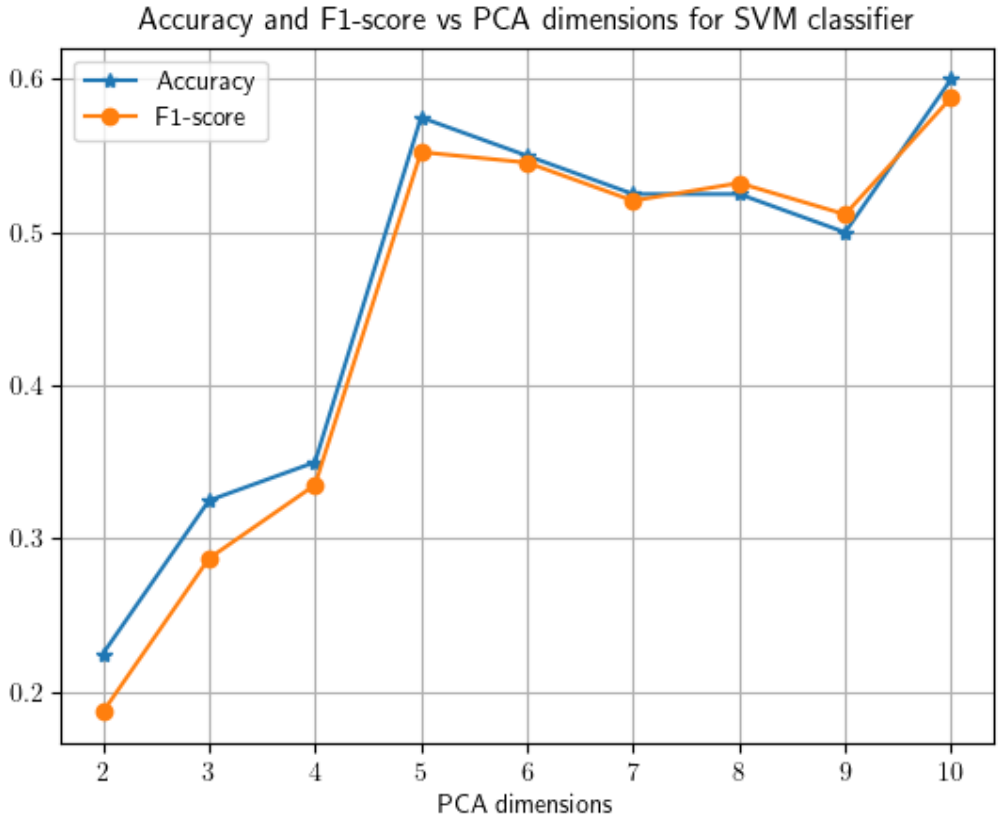
Στο βήμα αυτό, αρχικά χωρίζουμε τα δεδομένα μας σε δεδομένα εκπαίδευσης (train, 70%) και δεδομένα ελέγχου (test, 30%) με χρήση της συνάρτησης train_test_split. Προτού δοκιμάσουμε οποιοδήποτε μοντέλο κανονικοποιούμε τα δεδομένα για να τα χειρίζονται "δίκαια" μέθοδοι επιλογής χαρακτηριστικών όπως η PCA (που χρησιμοποιούμε ξανά) και μοντέλα ταξινόμησης που βασίζονται σε μετρικούς χώρους.

Ακολούθως, σε όλα τα μοντέλα μας θα δοκιμάζουμε ως υπερπαράμετρο το πλήθος συνιστωσών του PCA, από 2 έως και 10. Όλα τα μοντέλα θα αξιολογούνται στο τέλος με βάση το accuracy και το F1-score. Το F1-score το χρησιμοποιούμε κυρίως σε περιπτώσεις που υπάρχουν ανισορροπίες στην εκπροσώπηση των κλάσεων στα δεδομένα, κάτι που παρατηρούμε ότι εδώ δεν συμβαίνει. Για αυτό, σε όλα τα μοντέλα που εξετάζουμε, το accuracy και το f1-score είναι πολύ κοντά και έχουν την ίδια τάση.

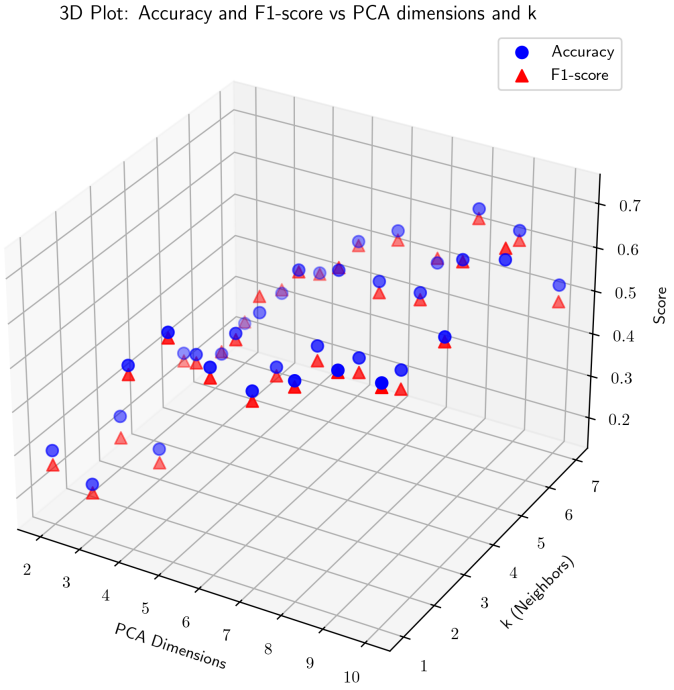
Το πρώτο μοντέλο που δοκιμάζουμε είναι ο Naive Bayes για Γκαουσιανές τυχαίες μεταβλητές (`GaussianNB()`). Παραθέτουμε το διάγραμμα των μετρικών ως προς το πλήθος συνιστωσών κατά το PCA. Το καλύτερο μοντέλο λήφθηκε για 10 συνιστώσες με ακρίβεια 60%.



Το δεύτερο μοντέλο που δοκιμάζουμε είναι το Support Vector Machine με το μη γραμμικό απειροδιάστατο RBF kernel (`SVC()`). Παραθέτουμε και εδώ το αντίστοιχο διάγραμμα. Το καλύτερο μοντέλο λήγθηκε πάλι για 10 συνιστώσες με ακρίβεια 60%.

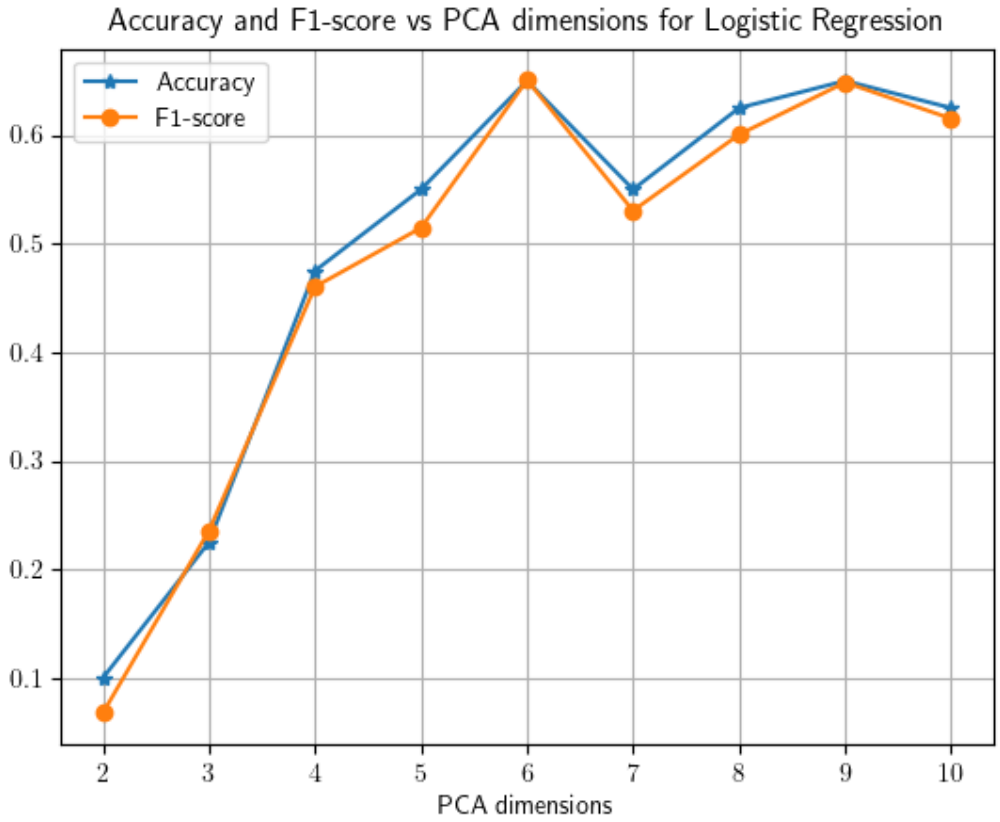


Το τρίτο μοντέλο που δοκιμάζουμε είναι το k-Nearest Neigbors, με επιπλέον υπερπαράμετρο το $k \in \{1, 3, 5, 7\}$ (`KNeighborsClassifier(n_neighbors=k)`). Παραθέτουμε το αντίστοιχο τριδιάστατο διάγραμμα. Το καλύτερο μοντέλο λήφθηκε για 10 συνιστώσες και k = 5 με ακρίβεια 70%.



Το τελευταίο μοντέλο που δοκιμάζουμε είναι το Logistic Regression (`LogisticRegression()`). Παρα-

θέτουμε πάλι το αντίστοιχο διάγραμμα. Το καλύτερο μοντέλο λήφθηκε για 6 συνιστώσες με ακρίβεια 65%.

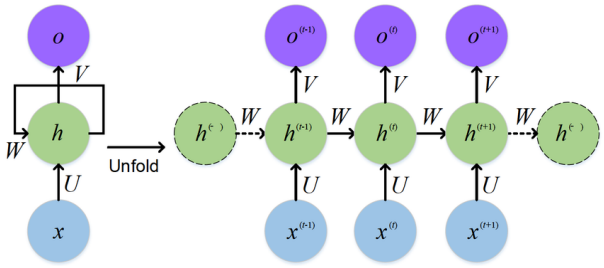


Βήμα 8

Σε αυτό το βήμα μας ζητείται να δημιουργήσουμε ακολουθίες 10 σημείων ενός ημιτόνου και ενός συνημιτόνου συχνότητας 40Hz. Στόχος είναι να προβλέψουμε τις ακολουθίες συνημιτόνου έχοντας ως δεδομένες εκείνες των ημιτόνων.

Για να το πετύχουμε αυτό επιστρατεύουμε 3 είδη νευρωνικών δικτύων, το RNN, το LSTM και το GRU. Για να τα ορίσουμε και να τα εκπαιδεύσουμε αξιοποιούμε την PyTorch.

- **RNN:** Πρόκειται για ένα τύπο νευρωνικού δικτύου ειδικά σχεδιασμένου ώστε να επεξεργαστούμε ακολουθιακά δεδομένα και να ανιχνεύσουμε τις εξαρτήσεις τους και τα μοτίβα τους στο χρόνο.



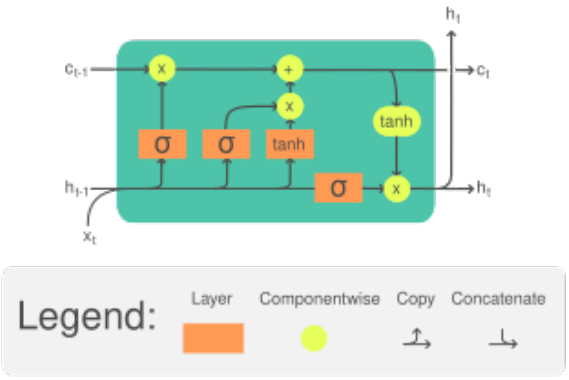
Ο κώδικας του ορισμού του είναι ο εξής:

```python
class myRNN(nn.Module):
    def __init__(self, input_size=1, hidden_size=32, output_size=1):
        super(myRNN, self).__init__()
        self.rnn = nn.RNN(input_size=input_size, hidden_size=hidden_size, batch_first=True)
        self.fc = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        out, _ = self.rnn(x)
        out = self.fc(out)
        return out
```

Ουσιαστικά έχουμε το component που υλοποιοεί ένα RNN, το οποίο πρσφέρεται έτοιμο από την PyTorch και κατόπιν ένα fully connected layer το οποίο συνδυάζει τα outputs των hidden layers του RNN κάθε χρονική στιγμή-δείγμα και μας δίνει το τελικό προβλεπόμενο output για τη δεδομένη χρονική στιγμή.

- **LSTM:** Το LSTM είναι ένα είδος RNN το οποίο αναπτύχθηκε προκειμένου να αντιμετωπίσει το πρόβλημα του vanishing gradient που εμφανιζόταν έντονα στον πρόγονό του. Ουσιαστικά, δίνει δυνατότητα short term memory στο μοντέλο ώστε να "θυμάται" πληροφορίες και εξαρτήσεις στα δεδομένα για χιλιάδες δείγματα, σε αντίθεση με το RNN το οποίο "έχανε" τις εξαρτήσεις δεδομένων που απείχαν κάποιο μεγάλο αριθμό δειγμάτων (αυτό είναι το manifestation του vanishing gradient).

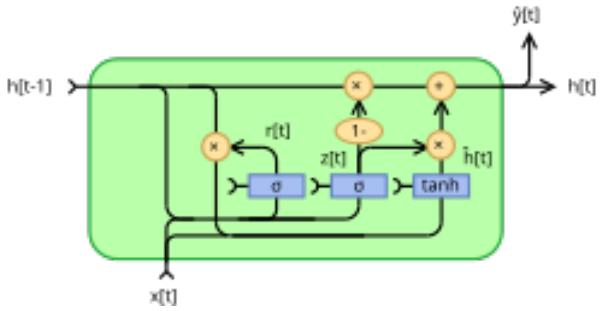


Ο κώδικας του ορισμού του φαίνεται παρακάτω και ακολουθεί τη λογική του RNN:

```python
class myLSTM(nn.Module):
    def __init__(self, input_size=1, hidden_size=32, num_layers=1, output_size=1): # hidden_size = 32 for moderate expressiveness
        super(myLSTM, self).__init__()
        self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first=True) # input is in format (batch_size, seq_len, input_size)
        self.fc = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        out, _ = self.lstm(x)
        out = self.fc(out)
        return out
```

- **GRU:** Είναι και αυτό "απόγονος" του RNN. Πρακτικά μοιάζει με το LSTM, μόνο που έχει έναν μηχανισμό από πύλες στο input, αλλά δεν έχει πύλη στο output και συνεπώς έχει λιγότερες παραμέτρους από το LSTM.



Ο κώδικας του ορισμού του φαίνεται παρακάτω και ακολουθεί τη λογική του RNN και του LSTM:

```
class myGRU(nn.Module):
    def __init__(self, input_size=1, hidden_size=32, output_size=1):
        super(myGRU, self).__init__()
        self.gru = nn.GRU(input_size=input_size, hidden_size=hidden_size,
    batch_first=True)
        self.fc = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        out, _ = self.gru(x)
        out = self.fc(out)
        return out
```

Στην συνέχεια, εκπαιδεύουμε τα μοντέλα μας και συγκρίνουμε την απόδοσή τους. Η διαδικασία που ακολουθούμε είναι η εξής:

1. **Δημιουργία δειγμάτων ημιτόνων και συνημιτόνων:**
   Με την NumPy δημιουργούμε τα σήματα-εισόδους για τα νευρωνικά δίκτυα μας καθώς και τα σήματα-στόχους που θέλουμε να προβέψουμε (t = np.linspace(0, 1, total_points), x = np.sin(2*np.pi*f*t), y = np.cos(2*np.pi*f*t))

2. **Εκπαίδευση των μοντέλων:**

```
def train_model(model, X_train, Y_train, epochs=100, lr=0.001):
    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr) # default
    learning rate small to avoid overshooting
    losses = []

    for epoch in range(epochs):
        model.train() # put model in trainning mode
        optimizer.zero_grad()   # clean accumulated gradients

        outputs = model(X_train)
        loss = criterion(outputs, Y_train)

        loss.backward()   # backpropagate loss gradients
        optimizer.step()  # update weights

        losses.append(loss.item())

        if (epoch + 1) % 20 == 0:
            print(f'Epoch [{epoch+1}/{epochs}], Loss: {loss.item():.6f}')

    return losses
```

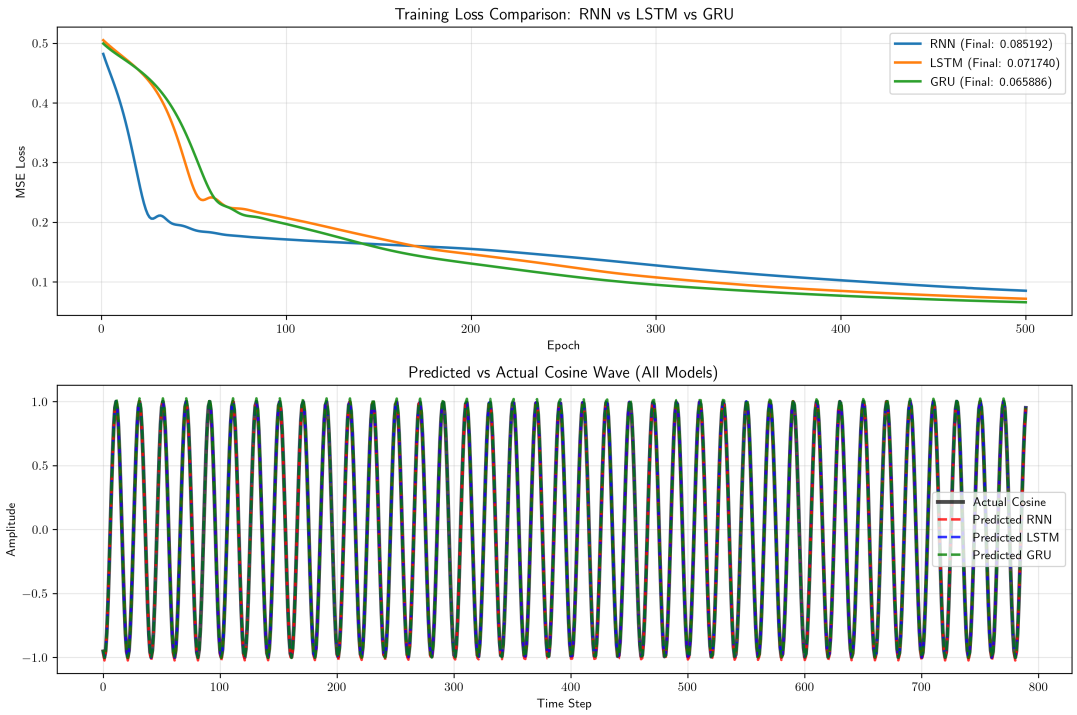
Ως κριτήριο ελαχιστοποίησης επιλέξαμε το Mean Square Error ενώ για optimizer τον Adam. Ο Adam ενσωματώνει και όρους ορμής στην δική του υλοποίηση του gradient descend. Εκπαιδεύουμε όλα τα μοντέλα για 100 εποχές.
Αρχικά, πριν από κάθε εποχή πρέπει να καθαρίσουμε τα προηγούμενα gradients γιατί η default συμπεριφορά της PyTorch είναι να τα συσσωρεύει, όχι να τα πανωγράφει.
Στη συνέχεια, υπολογίζουμε το output του μοντέλου και το loss (αντίστοιχο της απόκλισης από το στόχο μας). Τέλος κάνουμε backpropagation τα loss gradients και ενημερώνουμε τα βάρη.
Τα losses αποθηκεύονται ώστε να χρησιμοποιηθούν σε διαγράμματα μετέπειτα.

3. **Αξιολόγηση των μοντέλων:**

```
def evaluate_model(model, X_test, Y_test):
    model.eval()    # put model in eval mode
    with torch.no_grad():   # we do not need to calculate and store
    gradients in inference
        predictions = model(X_test)
        mse = nn.MSELoss()(predictions, Y_test)
        mae = torch.mean(torch.abs(predictions - Y_test))
    return predictions, mse.item(), mae.item()
```

Αρχικά, βάζουμε το μοντέλο σε eval mode (από documentation της PyTorch) και κατόπιν απενεργοποιούμε τον υπολογισμό και την αποθήκευση των gradients διότι αυτή η υπολογιστικά ακριβή διαδικασία δεν είναι απαραίτητη κατά το inference. Τέλος υπολογίζουμε τις μετρικές μας για τα test samples και τις επιστρέφουμε.

Αποτελέσματα



Σε αυτό το γράφημα εξετάζουμε τη συμπεριφορά των μοντέλων κατά την εκπαίδευσή τους και παίρνουμε μια πρώτη εικόνα για το κατά πόσο το αποτέλεσμα είναι ικανοποιητικό.

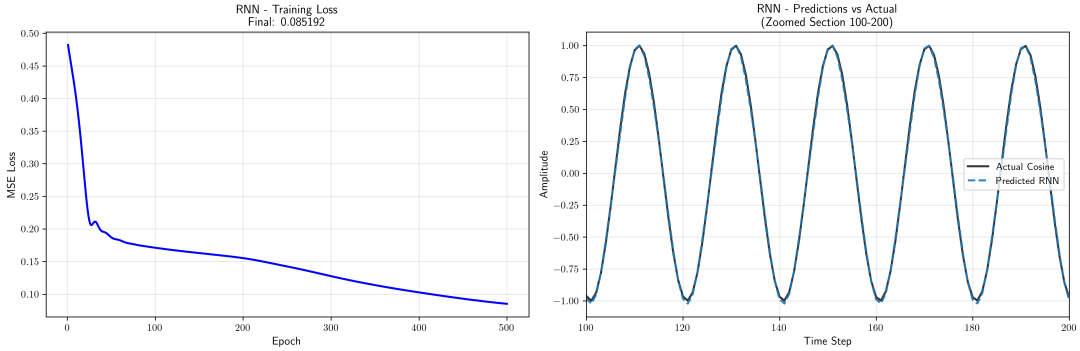
Βλέπουμε ότι έχουμε εκπαιδεύσει τα μοντέλα μας για 500 εποχές. Καταλήξαμε σε αυτή την επιλογή ως εξής:

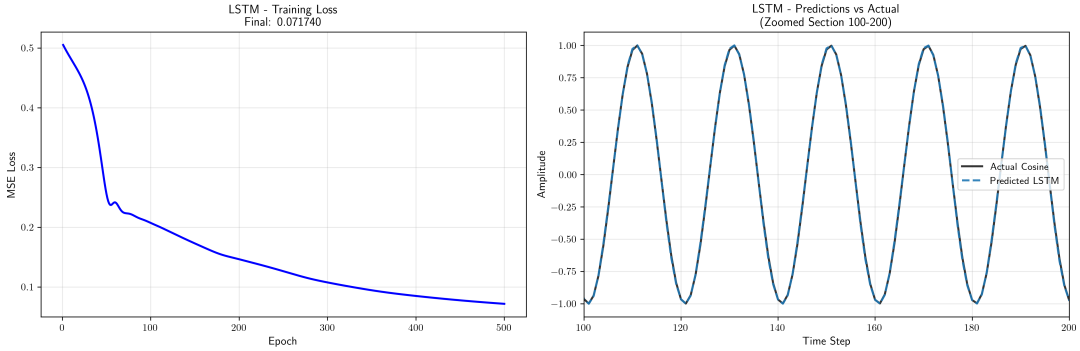
1. Αρχικά, εκπαιδεύσαμε με 100 εποχές. Αυτό οδήγησε στο εξής παράδοξο: Το RNN, θεωρητικά το πιο "αδύναμο" μοντέλο είχε το μικρότερο loss - δηλαδή είχε προσαρμοστεί καλύτερα στα δεδομένα. Αυτό μπορεί να έχει μόνο μία εξήγηση, δεδομένου ότι έχει μικρότερη χωρητικότητα από τα προηγούμενα 2. Ο μεγαλύτερος αριθμός παραμέτρων των LSTM και GRU δεν τους έχει επιτρέψει να συγκλίνουν ακόμα. Οπότε αυξάνουμε τον αριθμό εποχών σε 300.

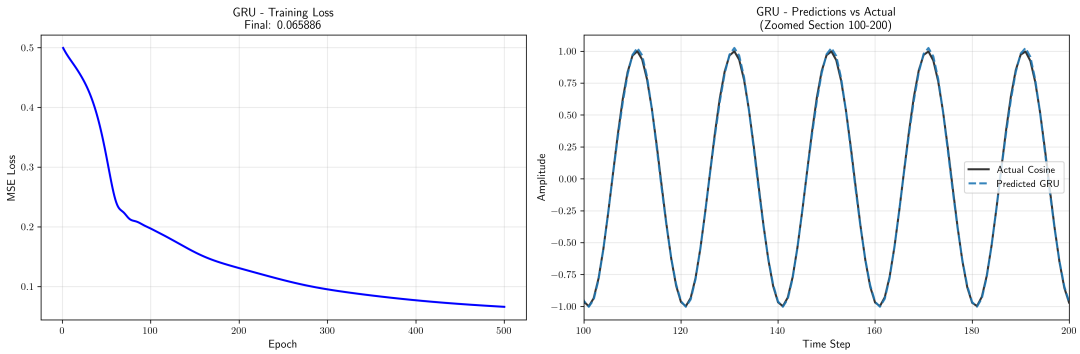
2. Όπως φαίνεται και στο plot παραπάνω, μετά τις 180 εποχές πράγματι τόσο το LSTM όσο και το GRU έχουν μικρότερο loss από το RNN. Αλλά βλέπουμε ότι το loss έχει την τάση να μειωθεί κι άλλο. Οπότε αυξάνουμε σε 500 εποχές

3. Βλέπουμε ότι το loss δεν βελτιώνεται οπότε σταματάμε (σε αυτό το σημείο θα μπορούσαμε να αρχίσουμε να αυξάνουμε τα hidden layers).

Αφότου εξηγήσαμε την διαδικασία της εκπαίδευσής μας, βλέπουμε ότι και το αποτέλεσμα για τα μοντέλα μας, οπτικά, είναι αρκετά υποσχόμενο. Για να το επιβεβαιώσουμε πλήρως εξετάζουμε το αποτέλεσμα σε ένα zoom της κυματομορφής για κάθε μοντέλο ξεχωριστά.







Και για τα 3 μοντέλα το αποτέλεσμα είναι πολύ ικανοποιητικό.

Τέλος, επιλέγουμε το καλύτερο μοντέλο με βάση το μικρότερο MSE στο test set. Αυτό βλέπουμε ότι είναι το GRU.

```
============================================================
BEST MODEL SELECTION
============================================================
1. GRU: MSE=0.0651, MAE=0.1122
2. LSTM: MSE=0.0712, MAE=0.1169
3. RNN: MSE=0.0847, MAE=0.1426

Best Model: GRU
============================================================
```

Βήμα 9

Πρώτα από όλα κάνουμε clone τα δεδομένα σε μορφή .wav από το repository της εργασίας στον υπολογιστή μας.

Κατόπιν, υλοποιούμε τη συνάρτηση *parse_free_digits* προκειμένου να διαβάσουμε σωστά τα αρχεία με τη βιβλιοθήκη librosa.

```python
def parse_free_digits(directory):
    # Parse relevant dataset info
    files = glob(os.path.join(directory, "*.wav"))
    fnames = [f.split("/")[1].split(".")[0].split("_") for f in files]
    ids = [f[2] for f in fnames]
    y = [int(f[0]) for f in fnames]
    speakers = [f[1] for f in fnames]
    _, Fs = librosa.core.load(files[0], sr=None)

    def read_wav(f):
        wav, _ = librosa.core.load(f, sr=None)

        return wav

    # Read all wavs
    wavs = [read_wav(f) for f in files]

    # Print dataset info
    print("Total wavs: {}. Fs = {} Hz".format(len(wavs), Fs))

    return wavs, Fs, ids, y, speakers
```

Το αποτέλεσμα της κλήσης της συνάρτησης είναι:

Total wavs: 3000. Fs = 8000 Hz

Στην συνέχεια βασισμένοι στην υπολοποίηση του βοηθητικού κώδικα ορίζουμε τη δικιά μας συνάρτηση *split_free_digits* την οποία παρουσιάζουμε παρακάτω:

```python
def split_free_digits(frames, ids, speakers, labels):
    print("Splitting in train test split using the default dataset split")
    # Split to train-test
    X_train, y_train, spk_train = [], [], []
    X_test, y_test, spk_test = [], [], []
    test_indices = {
        "0" : 0,
        "1" : 0,
        "2" : 0,
        "3" : 0,
        "4" : 0,
```

```
        "5" : 0,
        "6" : 0,
        "7" : 0,
        "8" : 0,
        "9" : 0
    }

    for idx, frame, label, spk in zip(ids, frames, labels, speakers):
        if str(idx) in test_indices.keys():
            X_test.append(frame)
            y_test.append(label)
            spk_test.append(spk)
            test_indices[idx] = test_indices[idx] + 1
        else:
            X_train.append(frame)
            y_train.append(label)
            spk_train.append(spk)

    print(f"Train dataset's size is {len(X_train)}") # <- DEBUGGING
    for key, values in test_indices.items():
        print(f"Train dataset has {values} samples of digit {key}")
        print(f"Train dataset has {values} samples of digit {key}")
    return X_train, X_test, y_train, y_test, spk_train, spk_test
```

Τρέχουμε τον κώδικα για να επιβεβαιώσουμε την ορθή λειτουργία του:

```
└─ python3 lab.py
Total wavs: 3000. Fs = 8000 Hz
Splitting in train test split using the default dataset split
Train dataset's size is 2400
Train dataset has 60 samples of digit 0
Train dataset has 60 samples of digit 0
Train dataset has 60 samples of digit 1
Train dataset has 60 samples of digit 1
Train dataset has 60 samples of digit 2
Train dataset has 60 samples of digit 2
Train dataset has 60 samples of digit 3
Train dataset has 60 samples of digit 3
Train dataset has 60 samples of digit 4
Train dataset has 60 samples of digit 4
Train dataset has 60 samples of digit 5
Train dataset has 60 samples of digit 5
Train dataset has 60 samples of digit 6
Train dataset has 60 samples of digit 6
Train dataset has 60 samples of digit 7
Train dataset has 60 samples of digit 7
Train dataset has 60 samples of digit 8
Train dataset has 60 samples of digit 8
Train dataset has 60 samples of digit 9
Train dataset has 60 samples of digit 9
```

Βλέπουμε ότι το split που έχουμε κάνει έχει μέγεθος 2400, δηλαδή είναι πράγματι το 80% του 3000. Επιπλέον, είναι ισορροπημένο, αφού υπάρχει ίδιος αριθμός δειγμάτων ήχου σε κάθε κατηγορία - ψηφίο (60). Επομένως ως μετρική του *accuracy* θα είναι αρκετό για να αξιολογήσουμε την επίδοση των ταξινομητών μας (όπως είδαμε και στην προπαρασκευή, στην περίπτωση του ισορροπημένου dataset το F1-score ακολουθεί το ίδιο trend με το accuracy).

References

[1] Lawrence R. Rabiner and Ronald W. Schafer. *Digital Processing of Speech Signals*. Prentice Hall, Upper Saddle River, NJ, 1978.